

1. SYNOPSIS

1.1 Title of the Project

“ElevateHire: Placement training and Networking system”

1.2 Objective of the Project

The objective of this project is to design and develop a unified web platform that connects students, colleges, and recruiters for end-to-end placement lifecycle management. The system aims to streamline resume creation, opportunities posting, assessment, and certification validation while providing transparent interaction among all stakeholders.

The platform is designed to:

- Automate eligibility filtering and candidate shortlisting.
- Provide AI-assisted resume analysis and job matching insights.
- Enable structured and secure communication between recruiters and candidates.
- Offer performance analytics dashboards for institutions.
- Establish a verified digital professional profile system for students.
- Reduce manual administrative workload in campus placement processes.

1.3 Project Category

Web Application / Educational Technology (EdTech) / Human Resources Technology (HRTech)

1.4 Software and Hardware Requirements

Software Requirements

- Operating System: Windows 10/11, Linux or macOS
- Frontend: Flutter Web (Dart)
- Backend: Supabase Functions (TypeScript) or equivalent serverless platform
- Database: PostgreSQL
- IDE/Tools: VS Code / Android Studio, Git & GitHub, Web Browser (Chrome/Edge)

- External Services: Certification Provider APIs

Hardware Requirements

- Development Machine: Intel i5/Ryzen 5, 8 GB RAM (16 GB recommended), 256 GB SSD, broadband internet
- Client Devices: Desktop/Laptop/Tablet/Smartphone with stable internet connection

1.5 Structure of the Program

1.5.1 Analysis

Traditional placement workflows are fragmented across spreadsheets, emails and offline records, causing difficulty in tracking applicants, validating achievements and aligning candidates with the right openings. The proposed system provides an integrated platform where students manage profiles and preparation, colleges control academic and placement activities, and recruiters access verified candidate data with minimal manual intervention. A centralized feed, structured modules and role-based access reduce redundancy, improve transparency and support data-driven decision-making during the placement process.

1.5.2 Data Structures

- User & Role Entities: Tables for Students, College Admins and Recruiters linked through role-based access control.
- Profiles & Resumes: Structured records storing academic details, experience, skills and AI-generated resume templates.
- Posts & Feed: Entities for announcements, achievements, opportunities and interactions forming a social-style feed.
- Assessments & Question Bank: Tables for aptitude/technical tests, questions (including AI-generated items) and attempts.
- Certifications: Records for uploaded certificates with category tags, issuing authority, validity and verification status.
- Jobs & Applications: Structures for college-posted and recruiter-posted opportunities and corresponding student applications.

1.5.3 Module Description

1. Student Module

The Student Module acts as the central workspace for managing academic and professional development. It includes profile management, achievement posting, and an integrated resume builder with AI-assisted suggestions and job match insights.

Students can access aptitude practice tests, apply to opportunities, and track applications in real time. They can enroll in courses offered by colleges and recruiters through the Learning Hub and earn certifications to enhance their profiles.

The module also provides certification management with validation status, a personal analytics dashboard for performance tracking, a Mock Interview System with structured feedback, and a Practice Centre to improve communication, technical, and interview skills.

2. College Module

The College Module enables institutional administrators to manage placement operations, academic coordination, and student performance tracking within a centralized system.

It includes:

- Creation and management of departments, student batches, and placement cycles.
- Posting campus drives, internships, projects, and internal opportunities through the Campus Feed.
- Moderation and management of the Campus Feed to ensure structured institutional communication.
- Creation of personalized learning content and institution-specific courses to support student skill development.
- Verification of student certifications category-wise (technical, soft skills, MOOCs, etc.) with structured approval workflows.
- Design and evaluation of mock tests, practice sets, and interview assessments.
- Question bank management including adding, updating, and organizing questions with expected keywords.
- Monitoring student performance in assessments, mock interviews, and applications through analytics dashboards.

- Tracking placement statistics and generating institutional performance reports.

3. Recruiter Module

The Recruiter Module enables secure onboarding, company profile creation, and structured recruitment management within the platform.

Recruiters can post job and internship opportunities, search and filter candidates based on skills, certifications, assessment scores, and resume data, and review category-validated certifications to improve trust in shortlisting decisions.

The module also allows recruiters to offer skill-based courses and learning programs to students, contributing to industry-aligned training. Recruiters can publish updates, hiring announcements, and opportunity-related posts through the platform feed to engage directly with eligible student groups.

It includes Recruiter AI Insights, an intelligent resume analysis system that evaluates how well a candidate's resume matches a specific job role. The system extracts resume content, prioritizes recruiter-defined requirements, and generates a weighted match score. Recruiters can instantly analyze applications, view match percentages, and identify matched and missing skills to support faster, data-driven hiring decisions. Real-time application statistics further enhance recruitment efficiency.

4. Intelligent Conversational Assistant

ElevateAI is the platform's intelligent conversational layer designed to act as a 24/7 career assistant that bridges static data and actionable guidance.

The system is role-aware and dynamically adapts based on the logged-in user. For students, it provides mentorship support such as learning recommendations, job discovery, and mock test guidance. For recruiters, it assists with job posting, candidate search, and recruitment management. For college administrators, it supports performance tracking and certification verification workflows.

The assistant operates using a heuristic navigation engine that enables actionable responses. Instead of only providing text-based guidance, it can trigger deep navigation within the platform, redirect users to relevant modules, and use confirmation flows for sensitive actions.

ElevateAI is integrated with the platform database to enable live data retrieval, including entity search (profiles, organizations, courses), recent job postings, and sample assessment questions. It also provides structured procedural guidance for platform-specific tasks such as certificate verification.

The current Beta version uses a high-performance local heuristic parser to ensure fast, secure, and platform-aligned responses. Future enhancements may integrate advanced large language models through server-side functions to enable broader conversational capabilities while maintaining structured action handling.

1.6 Limitations

- The platform requires continuous internet connectivity for real-time assessments, feed updates, AI analysis, and data synchronization.
- AI-based job matching and resume insights rely on keyword and heuristic analysis, which may not fully capture contextual or domain-specific expertise.
- Certification validation is limited to supported providers and predefined categories; some credentials may require manual verification.
- The current ElevateAI assistant (Beta) operates on a heuristic engine and may have limited conversational depth compared to full-scale language models.
- Large-scale deployment across multiple institutions may require advanced cloud scaling and performance optimization.

1.7 Future Scope of the Project

1. Integration of advanced AI (LLMs) for resume analysis and career guidance.
2. Implementation of in-platform video interview system.
3. Blockchain-based certification verification for security.
4. Expansion into corporate training and employee upskilling.

2 SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

2.1 Introduction

The Software Requirements Specification (SRS) document provides a detailed description of the ElevateHire platform. It defines the functional and non-functional requirements, technical specifications, system architecture, constraints, and expected behavior of the system.

This document acts as a reference framework for developers, testers, administrators, and stakeholders to ensure that the implementation aligns with the core objective of building an AI-enhanced placement training and networking ecosystem that bridges academia and industry.

2.2 Purpose

The purpose of ElevateHire is to provide a centralized, AI-driven placement management system where:

- Students can build verified digital professional identities and enhance career readiness.
- Colleges can manage placement workflows, validate credentials, and monitor performance analytics.
- Recruiters can identify verified talent and automate resume screening using AI-based insights.

The system replaces fragmented, manual placement processes with a structured, automated, and data-driven platform.

2.3 Scope

ElevateHire is a full-stack web application with modular architecture and role-based access control.

The scope includes:

- Student, College, and Recruiter dashboards
- AI-Powered Resume Matching Engine
- ElevateAI Conversational Assistant (Beta)
- Certification Validation and Trust System

- Mock Interview and Practice Center
- Learning Hub with course enrollment and certification generation
- Campus Feed and Professional Networking System
- Analytics and Reporting Dashboard

The system is designed to support scalable deployment across multiple institutions and recruitment organizations.

2.4 Functional Requirements

Functional requirements define the expected operations and behaviors of the system.

2.4.1 User Authentication and Authorization

- Secure role-based access for Students, College Administrators, and Recruiters.
- Email-based registration and verification.
- JWT-based session handling.
- Role-Level Access Control (RLAC) with Row-Level Security policies.

2.4.2 Student Module

- Profile management including academic records, skills, projects, internships, and achievements.
- AI-assisted resume builder with structured templates.
- AI-powered Resume Matching and Job Insights system.
- Access to aptitude practice tests and AI-generated assessments.
- Mock Interview system with structured evaluation.
- Course enrollment from colleges and recruiters through the Learning Hub.
- Certification upload, validation tracking, and trust scoring.
- Application submission and real-time status tracking.
- Personal analytics dashboard displaying performance metrics and skill gaps.

- Networking system for professional connections.

2.4.3 College Module

- Bulk student data management and department-level organization.
- Creation and management of placement drives and institutional events.
- Campus Feed moderation and announcement system.
- Certification validation workflow with approval/rejection mechanisms.
- Mock test and question bank management.
- Creation of institution-specific learning content and courses.
- Analytics dashboards for placement tracking and performance reporting.

2.4.4 Recruiter & AI Insights Module

- Company onboarding and recruiter profile management.
- Job and internship lifecycle management (create, update, close).
- Advanced candidate search and filtering.
- Recruiter AI Insights for automated resume evaluation using weighted keyword analysis.
- Real-time match score generation and skill gap identification.
- Application-level analytics and hiring pipeline management.
- Course offering capability for industry-driven training programs.
- Opportunity and hiring announcements via platform feed.

2.4.5 ElevateAI Conversational Assistant

- Role-aware conversational support.
- Deep-link navigation within the platform.
- Entity search (profiles, organizations, courses).
- Live data retrieval (jobs, assessment questions).
- Structured task guidance for platform workflows.

2.5 Non-Functional Requirements

2.5.1 Performance

- Resume analysis and match score generation must complete within 2–3 seconds.
- Database queries must respond within acceptable latency thresholds under concurrent usage.

2.5.2 Scalability

- Backend infrastructure must support horizontal scaling.
- Modular service architecture to allow feature expansion.

2.5.3 Reliability

- Use of PostgreSQL transactions to maintain data integrity.
- Backup and restore mechanisms via Supabase.

2.5.5 Usability

- Responsive UI compatible with desktop, tablet, and mobile devices.
- Consistent Material Design 3 interface standards.
- Clear navigation and accessible interface structure.

2.6 Software Requirements

2.6.1 Frontend Technologies

- Framework: Flutter (Web/Desktop/Mobile)
- Language: Dart
- State Management: Riverpod
- UI Framework: Material Design 3
- Typography: Google Fonts

2.6.2 Backend Technologies

- Platform: Supabase (Backend-as-a-Service)
- Database: PostgreSQL (with JSONB support)
- Authentication: Supabase Auth (JWT-based)
- Serverless Functions: Deno-based Edge Functions

- File Storage: Supabase Storage

2.6.3 Development & Deployment Tools

- Version Control: Git and GitHub
- Frontend Hosting: Vercel
- Backend Hosting: Supabase Cloud
- Development Environment: Visual Studio Code, Flutter DevTools

2.7 System Architecture

ElevateHire follows a Modular Feature-Based Architecture consisting of:

- Presentation Layer: Flutter UI components organized by feature modules.
- Domain Layer: Business logic services such as ResumeMatchingService and AIAssistantNotifier.
- Data Layer: Repository pattern communicating with Supabase APIs.
- External Integration Layer: Edge Functions connecting to AI services and third-party APIs.

This layered separation ensures maintainability, scalability, and testability.

2.8 Assumptions and Constraints

2.8.1 Assumptions

- Uploaded resumes are text-searchable PDFs.
- Users have stable internet connectivity.
- Users operate on modern web browsers.

2.8.2 Constraints

- Performance may depend on Supabase free-tier resource limits unless upgraded.
- AI-based heuristic matching may not fully replace human judgment in hiring decisions.
- Real-time operations depend on cloud service availability.

3. SYSTEM DESIGN

3.1 Introduction

System Design defines the architecture, components, interfaces, and data flow mechanisms required to satisfy the system requirements specified in the SRS.

For ElevateHire, the design emphasizes:

- Modular feature-based architecture
- Secure role-based data access
- Real-time synchronization
- AI-assisted processing
- Scalability using serverless backend infrastructure

The system follows a layered architecture separating presentation, business logic, and data layers to ensure maintainability and extensibility.

3.2 System Architecture

3.2.1 High-Level Architecture

ElevateHire follows a modern three-tier architecture consisting of:

1. Presentation Layer (Flutter Web/Application)
2. Application & Platform Logic Layer (Supabase Services + AI Engine)
3. Data Layer (PostgreSQL with Row-Level Security)

Architectural Characteristics

- Secure communication through HTTPS and WebSocket protocols.
- JWT-based authentication and session handling.
- Row-Level Security (RLS) for enforcing role-based data isolation.
- Real-time updates for feeds, applications, and notifications.
- Serverless logic execution for AI processing and email communication.

This architecture ensures separation of concerns and supports scalable expansion.

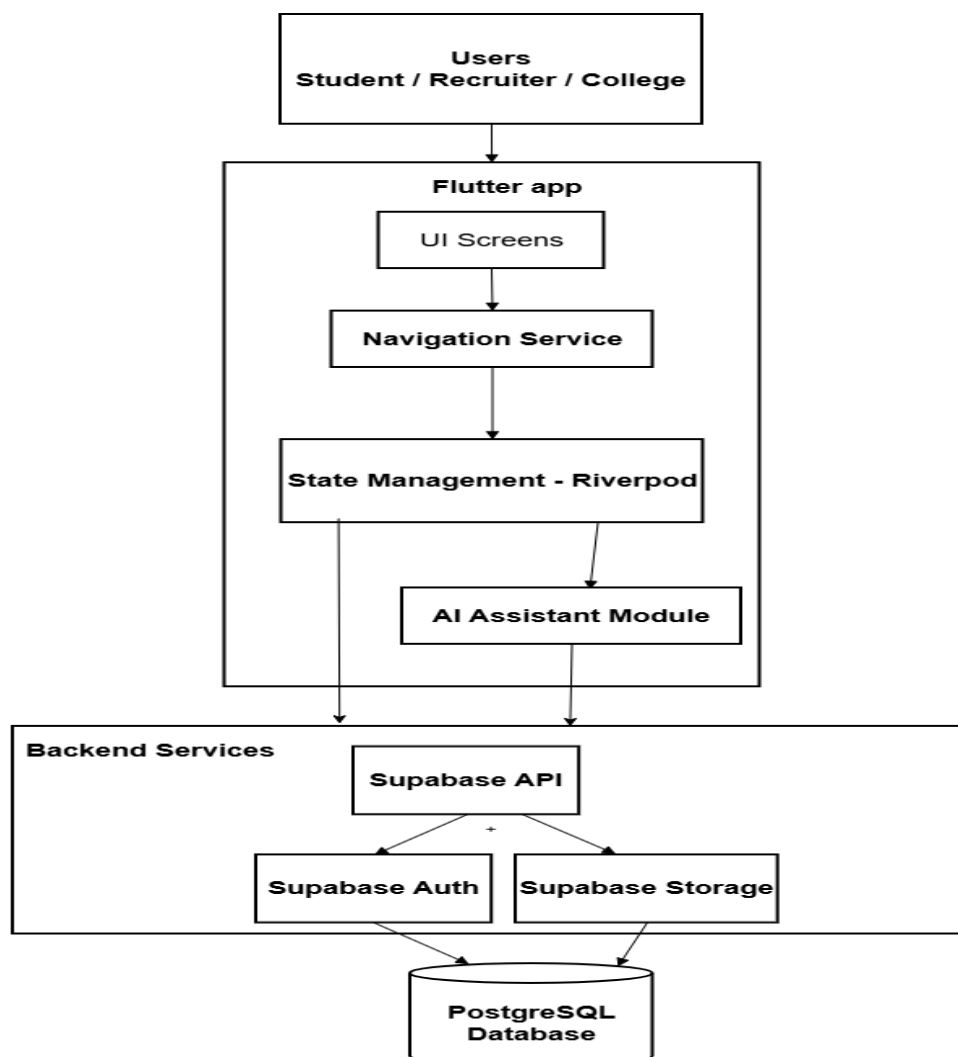


Figure 3.2.1: High-Level Architecture

3.3 Detailed System Design

3.3.1 Modules and Their Functionality

The system is divided into independent yet interoperable modules, each responsible for a specific domain of functionality.

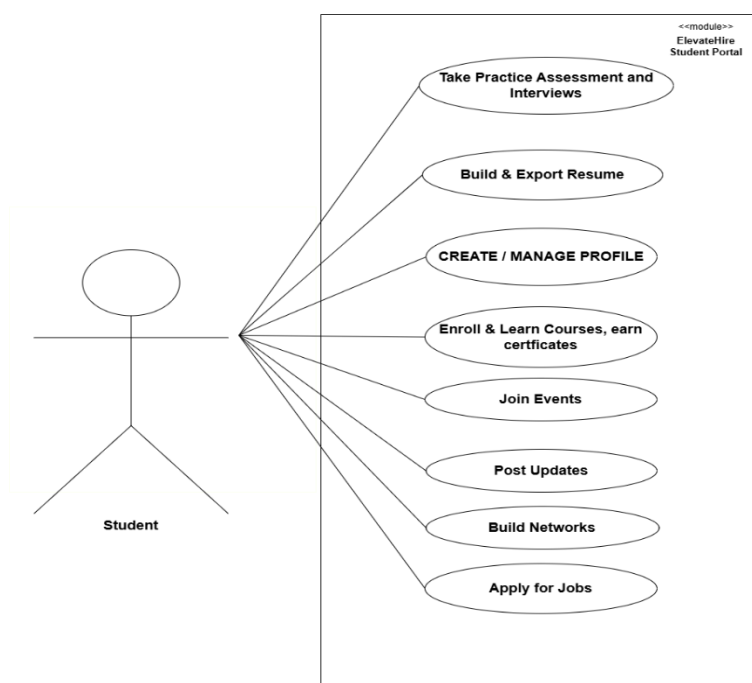
Module	Core Functionality	Key Components
Authentication	Secure user access and session control	Supabase Auth, JWT Tokens, Role-Based Routing

Student Portal	Professional profile management and skill development	Resume Builder, Practice Arena, Mock Interview
Recruiter Portal	Job lifecycle management and applicant screening	Job Management, AI Resume Insights
College Admin Module	Institutional placement oversight and validation	Placement Tracker, Certification Verification Engine
Verification Engine	Certificate authentication and trust scoring	Trust Score Calculator, Validation Rules
AI Assistant	Conversational navigation and contextual support	ElevateAI (Beta), Local Heuristic Parser
Networking System	Professional communication and updates	Connections, Real-time Messaging, Campus Feed

Table3.3.1: Modules and Their Functionality

3.4 Use Case Diagram

The use case diagram illustrates the interactions between system actors and core functionalities.



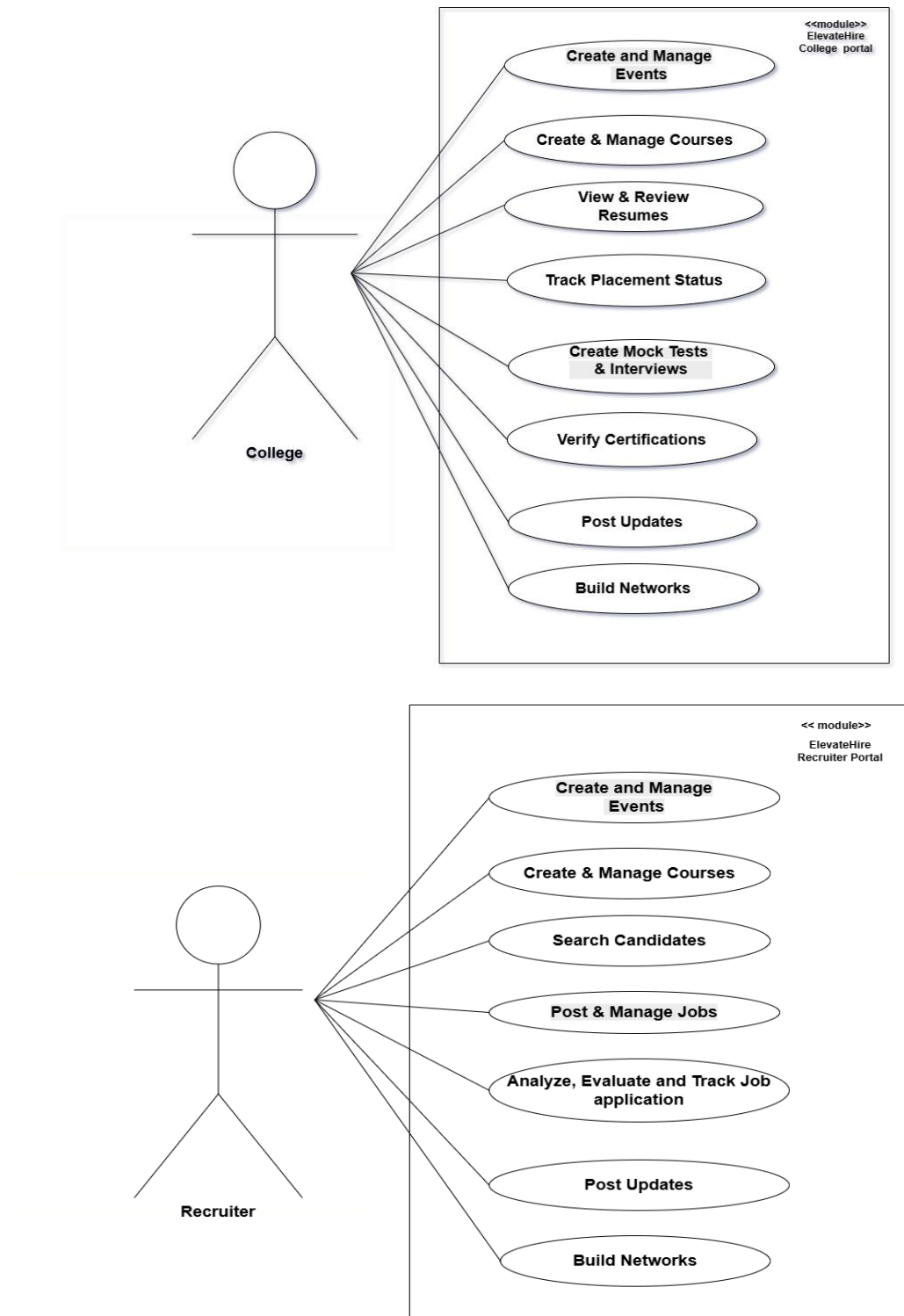


Figure 3.4: Use Case Diagram

4.DATABASE DESIGN

4.1 Introduction

The database design of ElevateHire is implemented using a normalized and modular PostgreSQL schema managed through Supabase. The schema emphasizes:

- Referential integrity through foreign key constraints
- Role-based data isolation via Row-Level Security (RLS)
- High concurrency support
- Scalable multi-tenant institutional management

The database is designed to support the complete lifecycle of placement training, recruitment workflows, certification validation, and real-time learning operations.

4.2 Database Overview

The ElevateHire database is structured into logical modules to ensure scalability, maintainability, and performance optimization. The schema consists of more than 70 tables, covering identity management, recruitment processes, learning systems, certification tracking, and engagement features.

Module Breakdown

- Identity & Institutional Module
Manages users, profiles (Student/Recruiter/College), and organizational hierarchies such as departments and programs.
- Recruitment Lifecycle Module
Handles job postings, interview round sequencing, submissions, evaluations, application tracking, and offer management.
- Skill & Validation Engine
Tracks certification validation workflows, skill scoring, and provider trust metrics.
- Academic & Aptitude Module
Manages learning courses, lecture structures, aptitude question banks, and assessment attempts.

- Engagement & Communication Module
Supports the professional feed, networking connections, messaging, and automated notification logs.

4.3 Database Tables Structure

The ElevateHire database consists of multiple relational tables organized into functional modules such as Identity Management, Recruitment, Learning, Certification, Communication, and Networking.

The following tables represent the core operational structure of the system.

4.3.1 ORGANIZATIONS

Column	Description
id (PK)	Unique organization identifier
name	Organization name
short_code	Unique organization code
type	College or Company
website	Website URL
address_city	City
address_state	State
address_country	Country
logo_url	Logo
created_by (FK)	References auth.users
created_at	Creation timestamp

Table 4.3.1: ORGANIZATIONS

4.3.2 PROFILES

Column	Description
id (PK, FK)	References auth.users
email	User email
role	Student / Recruiter / College
full_name	User full name
phone	Contact number
avatar_url	Profile image
organization_id (FK)	References organizations.id
bio	Profile description
linkedin_url	LinkedIn profile
github_url	GitHub profile
resume_url	Resume link
created_at	Account created time

Table 4.3.2: PROFILES

4.3.3 STUDENT_PROFILES

Column	Description
id (PK, FK)	References profiles.id
usn	University number
department_id (FK)	References college_departments.id
program_id (FK)	References college_programs.id
batch_id (FK)	References college_batches.id
c_gpa	Academic CGPA
semester	Current semester
placement_status	Seeking / Placed
placed_company	Company name
placed_package	Salary package

Table 4.3.3: STUDENT_PROFILES

4.3.4 JOBS

Column	Description
id (PK)	Unique job ID
recruiter_id (FK)	References profiles.id
organization_id (FK)	References organizations.id
title	Job title
description	Job description
requirements	Required skills
location	Job location
salary_range	Salary
job_type	Internship / Fulltime
status	Open / Closed
created_at	Posted time

Table 4.3.4: JOBS

4.3.5 JOB_APPLICATIONS

Column	Description
id (PK)	Unique application ID
job_id (FK)	References jobs.id
student_id (FK)	References profiles.id
resume_url	Submitted resume
cover_letter	Cover letter
status	Applied / Shortlisted / Selected
applied_at	Applied timestamp
overall_score	AI matching score

Table 4.3.5 JOB_APPLICATIONS

4.3.6 JOB_ROUNDS

Column	Description
id (PK)	Unique round ID
job_id (FK)	References jobs.id
round_number	Round order
round_type	Technical / HR / Coding
title	Round name
deadline	Submission deadline
is_mandatory	Mandatory round

Table 4.3.6: JOB_ROUNDS

4.3.7 OFFERS

Column	Description
id (PK)	Offer ID
application_id (FK)	References job_applications.id
package_amount	Salary
location	Job location
joining_date	Joining date
status	Extended / Accepted / Declined

Table 4.3.7: OFFERS

4.3.8 STUDENT_PROJECTS

Column	Description
id (PK)	Project ID
student_id (FK)	References profiles.id
title	Project title
description	Project details
technologies	Tech stack
github_url	GitHub link
project_url	Live link
created_at	Upload date

Table 4.3.8: STUDENT_PROJECTS

4.3.9 STUDENT_CERTIFICATIONS

Column	Description
id (PK)	Certification ID
student_id (FK)	References profiles.id
provider_id (FK)	References certification_providers.id
certificate_name	Certificate title
certificate_url	Certificate file
validation_status	Pending / Verified
skill_points_earned	Points earned

Table 4.3.9: STUDENT_CERTIFICATIONS

4.3.10 LEARNING_COURSES

Column	Description
id (PK)	Course ID
title	Course name
provider_name	Course provider
category	Course category
difficulty	Beginner / Intermediate
placement_relevance	High / Medium / Low
skill_points	Points gained

Table 4.3.10: LEARNING_COURSES

4.3.11 COURSE_ENROLLMENTS

Column	Description
id (PK)	Enrollment ID
course_id (FK)	References learning_courses.id
student_id (FK)	References auth.users
progress	Completion percentage
is_completed	Completion status
certificate_url	Certificate link

Table 4.3.11 COURSE_ENROLLMENTS

4.3.12 POSTS

Column	Description
id (PK)	Post ID
author_id (FK)	References profiles.id
content	Post content
image_urls	Attached media
post_type	General / Achievement
created_at	Posted time

Table 4.3.12 POSTS

4.3.13 MESSAGES

Column	Description
id (PK)	Message ID
sender_id (FK)	References auth.users
receiver_id (FK)	References auth.users
content	Message text
is_read	Read status
created_at	Sent time

Table 4.3.13: MESSAGES

4.3.14 EVENTS

Column	Description
id (PK)	Event ID
college_id (FK)	References organizations.id
title	Event title
event_type	Workshop / Drive
start_date	Start time
end_date	End time
venue	Location

Table 4.3.14: EVENTS

4.4 Normalization Strategy

The database follows Third Normal Form (3NF) principles:

1. Elimination of Data Redundancy
Repeated data such as organization details are stored in a centralized ORGANIZATIONS table.
2. Functional Dependency Control
Non-key attributes depend only on primary keys (e.g., job details depend solely on job_id).
3. Separation of Concerns
Identity, recruitment, learning, and certification systems are separated into modular entities.

This ensures consistency, maintainability, and performance optimization.

4.5 Key Constraints & Integrity Enforcement

The database ensures data consistency through:

- Primary Keys (UUID-based) for uniqueness.
- Foreign Key Constraints to enforce referential integrity.
- CHECK Constraints to validate enumerated fields (e.g., job status, difficulty levels).
- Unique Constraints for critical identifiers such as certificate numbers.
- Row-Level Security (RLS) policies to restrict unauthorized data access.

4.6 Security Model

Security is implemented at the database level using Supabase features:

- JWT-based authentication via Supabase Auth.
- Row-Level Security policies restricting data visibility by role.
- Secure file storage for resumes and certificates.
- Controlled access to sensitive tables such as job applications and evaluations.

This layered security approach ensures confidentiality and data isolation across institutions and recruiters.

4.7 Scalability & Performance Considerations

The schema is designed to support:

- High concurrency during assessments and recruitment drives.
- JSONB storage for semi-structured AI-related data.
- Indexed foreign keys for fast relational joins.
- Modular table expansion without redesigning core entities.

The use of PostgreSQL ensures robust transactional consistency and horizontal scalability.

4.8 Conclusion

The database design of ElevateHire demonstrates a modular, scalable, and secure relational architecture capable of handling complex placement workflows, AI-driven assessments, certification validation, and real-time engagement systems. The schema effectively balances normalization with flexibility, enabling both structured querying and semi-structured data storage.

5. DETAIL DESIGN

5.1 Introduction

The Detailed Design phase translates the high-level system architecture into a granular technical blueprint. For ElevateHire, this involves defining the internal logic of Flutter components, the state management lifecycle using Riverpod, and the structured interaction between frontend modules and Supabase backend services.

This chapter serves as the implementation reference for functional modules, defining service responsibilities, data flows, and modular boundaries.

5.2 System Overview

ElevateHire is implemented using a Feature-First Modular Architecture. Instead of separating code purely by technical layers (controllers, views, services), each functional domain (Authentication, Student, Recruiter, College, Networking) encapsulates its own:

- Presentation Layer
- Domain Layer
- Data Layer

This approach ensures high cohesion within modules and minimal coupling across features.

5.2.1 Key Technical Pillars

Presentation Layer

Built using Flutter with Material Design 3. Widgets are reactive and rebuild only when dependent Riverpod providers change state.

State Management

Managed by Riverpod using:

- StateNotifierProvider for synchronous state control
- AsyncNotifier for asynchronous database operations
- FamilyProvider for parameterized data fetching

Data Access Layer

Implements the Repository Pattern. UI components never directly access Supabase. All database interactions are encapsulated within repositories.

5.3 Module-wise Detailed Design

5.3.1 Authentication & Identity Module

Purpose

Secure session handling and role-based feature initialization.

Service Components

AuthRepository

Encapsulates Supabase authentication methods:

- Sign in
- Sign out
- Session refresh
- Token validation

RoleGuard

Middleware logic that validates profiles.role before route access.

IdentityHelper

Fetches extended metadata (organization_id, verification status, completion score).

Internal Flow

1. User submits login credentials.
2. AuthRepository validates via Supabase Auth (JWT issued).
3. IdentityHelper fetches corresponding profile record.
4. RoleGuard validates user role.
5. Application routes to:
 - Student Home
 - Recruiter Dashboard
 - College Panel

Data Interaction

- auth.users
- profiles
- organizations

Security Enforcement

- Row-Level Security (RLS)
- JWT-based authentication
- Route protection middleware

5.3.2 Student & Career Module

Purpose

Manage student digital identity, projects, resume intelligence, and assessment lifecycle.

Service Components

ResumeMatchingService

Implements keyword tokenization and coverage scoring between:

- Job.requirements
- Resume.text_content

ProjectService

Handles:

- Media uploads to Supabase Storage
- Feed post synchronization
- Project metadata persistence

AssessmentEngine

Manages:

- Timers
- Question randomization
- Score calculation
- Attempt persistence

Internal Flow

1. Student updates project details.
2. Media assets uploaded to Supabase Storage.
3. Database record created in student_projects.
4. Feed post auto-generated via Networking module.
5. Skill recalculation triggered using:
 - Extracted tech stacks
 - Assessment results

State Management

- studentProfileProvider
- studentProjectsProvider
- aptitudeAttemptProvider

Data Interaction

- student_profiles
- student_projects
- resumes
- aptitude_attempts
- student_skill_scores

5.3.3 Recruiter & Job Insights Module

Purpose

Enable structured job posting, automated screening, and applicant analytics.

Service Components

JobRepository

Handles transactional creation of:

- jobs
- job_rounds

ApplicantScanner

Executes batch scoring logic:

- Fetch new applications
- Run ResumeMatchingService
- Persist match_score

Internal Flow

1. Recruiter creates job with requirements.
2. Job stored in jobs.
3. Students apply via job_applications.
4. ApplicantScanner processes new applications.
5. Resume parsed locally.
6. Match Score computed.
7. Score persisted for recruiter dashboard filtering.

Data Interaction

- jobs
- job_applications
- job_rounds
- offers
- profiles

Optimization

- Index on job_id and student_id
- Precomputed match_score field
- Lazy-loaded application analytics

5.3.4 College & Placement Module

Purpose

Provide institutional placement monitoring and certification validation.

Service Components

VerificationService

Uses validation_rules and provider trust scores to determine:

- Auto approval
- Manual review
- Rejection

AnalyticsService

Executes aggregation queries:

- Placement percentage
- Average package
- Department-wise distribution

Internal Flow

1. Student uploads certification.
2. VerificationService checks:
 - Provider trust_score
 - Pattern match rules
3. If threshold met → Auto-verified
4. Else → Admin review queue
5. Upon approval:
 - Trust Score updated
 - Skill points recalculated

Data Interaction

- student_certifications
- validation_rules
- offers
- college_departments
- student_profiles

5.3.5 Networking Module

Purpose

Enable professional interaction, feed generation, and real-time social engagement.

Service Components

FeedService

Implements:

- Paginated fetching
- Sort by recency and relevance

InteractionManager

Handles:

- Likes
- Comments
- Connection requests
- Real-time updates via Supabase Realtime

Internal Flow

1. User creates post.
2. Post saved in posts.
3. FeedProvider updates via realtime listener.
4. Engagement metrics update counters incrementally.

Data Interaction

- posts
- post_likes
- post_comments
- connections
- messages

5.4 Modular Decomposition Components

To ensure maintainability, the system is decomposed into shared infrastructure components.

1. Shared UI Components
Located in lib/core/widgets.
Includes reusable buttons, cards, chips, and layout wrappers.
2. Navigation Service
Centralized route management.
Supports deep linking and role-based route protection.
3. Supabase Client Service
Singleton provider ensuring consistent backend connectivity.
4. Local Rule Engine
Parses natural language commands for ElevateAI Beta Assistant.

5.5 Software Package Structure

```
lib/  
├── core/           # Global utilities, themes, and shared widgets  
├── features/  
│   ├── auth/      # Login, Signup, Role logic  
│   ├── student/   # Profile, Resume, Projects, Assessment  
│   ├── recruiter/ # Job Posting, Applicant Tracking, Insights  
│   ├── college/   # Dashboard, Verification, Batches  
│   └── networking/ # Social feed, Connections, Messaging  
├── app.dart        # Main application entry point  
└── main.dart       # Initialization (Supabase, Riverpod)
```

Figure 5.5: Software Package Structure

Each feature contains:

- presentation/
- domain/
- data/

This enforces clean architectural separation.

5.6 System Structure Chart

ElevateHire follows a hierarchical modular architecture where the system is divided into major functional subsystems. Each subsystem is further decomposed into well-defined sub-functions to ensure clarity, scalability, and maintainability. The structure chart illustrates the top-down

decomposition of the platform into four primary portals, each responsible for a specific stakeholder role.

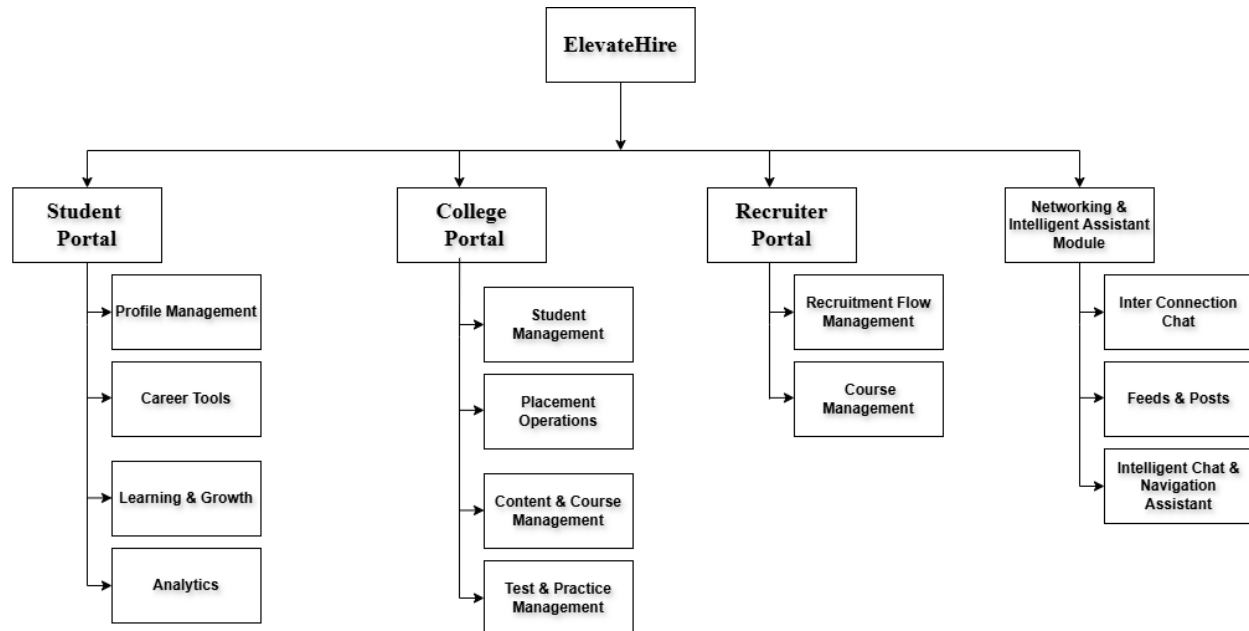


Figure 5.6: System Structure Chart

5.7 Error Handling Strategy

- Repository-level try-catch wrappers
- Standardized Failure objects
- Graceful UI fallback states
- Constraint-level validation at database

5.8 Scalability Considerations

- Multi-tenant organization model
- Indexed foreign keys
- Controlled Realtime subscriptions
- JSONB usage for semi-structured metadata
- Provider auto-dispose for memory optimization

6. CODING

6.1 Introduction

The coding phase of ElevateHire transforms system design specifications into fully functional software modules. This stage emphasizes modular architecture, clean code principles, structured state management, and scalable backend interaction.

The implementation is carried out using Flutter (Dart) for the frontend and Supabase (PostgreSQL, Auth, Storage, Realtime) for backend services. Special focus is given to maintainability, extensibility, and separation of concerns to support long-term evolution of the platform.

6.2 Programming Practices

6.2.1 Top-Down and Bottom-Up Approaches

The development of ElevateHire followed a hybrid strategy combining both bottom-up and top-down methodologies to ensure stability, flexibility, and efficient feature delivery.

Bottom-Up Approach:

The development began with constructing foundational components and core infrastructure. Low-level elements such as the Supabase client configuration, reusable UI components (buttons, input fields, layout containers), data models, and repository interfaces were implemented first. This approach ensured that higher-level modules were built upon a stable, reusable, and well-tested base architecture.

Top-Down Approach:

After establishing the core infrastructure, feature development followed a top-down methodology. High-level functional requirements were decomposed into smaller modules, such as the Student Resume Builder, Job Posting System, AI Resume Matching Engine, and Mock Interview System. User interface prototypes were created early, followed by gradual integration with backend services and business logic. This allowed rapid validation of workflows while maintaining architectural consistency.

The combination of both approaches enabled structured development while supporting iterative feature enhancement.

6.2.2 Structured Programming Principles

ElevateHire follows structured programming to improve clarity and reliability.

Modularity

Each feature is isolated inside a dedicated folder with presentation, domain, and data layers.

Single Responsibility Principle (SRP)

Each class performs one well-defined task:

- AuthRepository handles authentication.
- ResumeMatchingService handles resume scoring.
- VerificationService handles certification validation.

Controlled Flow Structures

Standard control constructs (if/else, switch-case, try/catch) ensure predictable execution and error handling.

6.2.3 Verification and Quality Assurance

Verification was embedded into development workflows.

- Static analysis using Dart Analyzer and lint rules.
- Type safety enforced via null safety.
- Isolated module testing using Flutter DevTools.
- Logging-based validation for AI match scoring and rule parsing.

6.3 Source Code Snippets

6.3.1 Authentication Repository

```
class AuthRepository {
    final SupabaseClient _supabase;

    AuthRepository(this._supabase);

    Future<AuthResponse> signInWithEmail({
        required String email,
        required String password,
    }) async {
        try {
            return await _supabase.auth.signInWithPassword(
                email: email,
                password: password,
            );
        } on AuthException catch (e) {
            throw Exception(e.message);
        }
    }
}
```

Figure 6.3.1: Authentication Repository Code Snippet

This repository abstracts Supabase authentication, ensuring the UI does not directly interact with backend APIs.

6.3.2 Resume Matching Service

```
class ResumeMatchingService {
    double calculateMatchScore(String resumeText, List<String> requirements) {
        if (requirements.isEmpty) return 0.0;

        final resumeTokens = resumeText.toLowerCase().split(RegExp(r'[\W_]+'));
        int matchCount = 0;

        for (var req in requirements) {
            if (resumeTokens.contains(req.toLowerCase())) {
                matchCount++;
            }
        }

        return (matchCount / requirements.length) * 100;
    }
}
```

Figure 6.3.2: Resume Matching Service Code Snippet

This logic performs:

- Keyword tokenization
- Requirement matching
- Weighted coverage calculation

The result is stored in `job_applications.overall_score`.

6.3.3 Application Entry Point

The main entry point initializes:

- Supabase backend connection
- Riverpod state container
- Global AI Assistant overlay

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Supabase.initialize(
    url: 'https://your-project.supabase.co',
    anonKey: 'your-anon-key',
  );
  runApp(const ProviderScope(child: ElevateHireApp()));
}

class ElevateHireApp extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    return MaterialApp(
      builder: (context, child) {
        // Global overlay for the AI Assistant sidekick
        return Stack(children: [child!, const AIAssistantWidget()]);
      },
      initialRoute: '/',
      routes: {
        '/': (context) => const AuthWrapper(),
        '/dashboard': (context) => const AppWrapper(),
      },
    );
  }
}
```

Figure 6.3.3: Application Entry Point Code Snippet

6.3.4 ElevateAI Heuristic Engine

```
class AIAssistantNotifier extends Notifier<AIAssistantState> {
  Future<void> _handleLocalRules(String text) async {
    final query = text.toLowerCase().trim();
    String reply = "";
    String? action;

    // Entity Search Logic (Profiles & Organizations)
    if (query.contains("who is") || query.startsWith("search ")) {
      final searchTerm = query.replaceAll(RegExp(r'who is|search'), '').trim();
      final profile = await _supabase.from('profiles')
        .select('full_name, role').ilike('full_name', '%$searchTerm%')
        .maybeSingle();

      if (profile != null) {
        reply = "I found ${profile['full_name']}. Open their profile?";
        action = "/network-profile?id=${profile['id']}";
      }
    }

    // Feature Mapping
    if (query.contains("practice") || query.contains("test")) {
      reply = "Opening the Practice Arena for you!";
      action = "/interview-prep";
    }

    state = state.copyWith(messages: [...state.messages, ChatMessage(text: reply)], lastAction: action);
  }
}
```

Figure 6.3.4: ElevateAI Heuristic Engine Code Snippet

This engine uses:

- Local heuristic parsing
- Entity lookup in PostgreSQL
- Deep linking for navigation
- Action confirmation logic

It operates without external LLM dependency in Beta mode

6.4 Error Handling Strategy

ElevateHire implements layered error handling:

1. Repository Layer: Catches Supabase exceptions.
2. Domain Layer: Validates business rules.
3. Presentation Layer: Displays user-friendly error messages.

All critical flows include null safety and fallback mechanisms.

6.5 Performance Optimization Techniques

- Lazy loading for feed and job listings.
- Pagination for large datasets.
- Indexed columns for job search and resume matching.
- Local caching of frequently accessed profile data.
- Efficient JSONB querying in PostgreSQL.

6.6 Security Considerations in Code

- JWT-based authentication.
- Role-based route guards.
- Row-Level Security (RLS) enforced at database level.
- Controlled file access via Supabase Storage policies.
- Input sanitization before database writes.

6.7 Future Enhancements in Coding Layer

The current system is modular and extendable. Future improvements may include:

1. Migration of Resume Matching to Supabase Edge Functions for distributed processing.
2. Integration of vector embeddings for semantic resume analysis.
3. Real-time interview scoring using AI speech evaluation.
4. Advanced analytics using materialized SQL views.
5. Native mobile build optimization for Android and iOS.
6. Event-driven architecture using Supabase triggers for automation workflows.

6.8 Conclusion

The coding structure of ElevateHire reflects a scalable, modular, and AI-integrated architecture. By combining Flutter's reactive UI system with Supabase's serverless backend, the platform ensures real-time performance, structured data integrity, and maintainable feature expansion.

The system is designed not only for current placement lifecycle automation but also for future intelligent career ecosystem development.

7. TESTING

7.1 Introduction

Testing is a critical phase in the ElevateHire development lifecycle, ensuring that the platform operates reliably, securely, and in alignment with defined requirements. The testing process was designed to validate system correctness, role-based security, AI-driven analytics accuracy, and database integrity across multiple modules.

This chapter outlines the testing objectives, methodologies, validation procedures, and representative test cases executed during development.

7.2 Testing Objectives

The primary objectives of testing were:

- **Functional Accuracy:** Validate correctness of resume matching, assessment scoring, and placement analytics.
- **Security Enforcement:** Ensure Row-Level Security (RLS) policies prevent unauthorized cross-role data access.
- **Data Integrity:** Confirm relational consistency across PostgreSQL tables.
- **Reliability:** Ensure real-time synchronization via Supabase Realtime operates without data inconsistency.
- **Performance:** Validate resume parsing and match score computation within acceptable response time.
- **Usability:** Ensure intuitive and responsive UI behavior across device types.

7.3 Testing Methods

The following testing methodologies were applied:

7.3.1 Manual Testing

Used for validating UI workflows, routing logic, role transitions, and visual components.

7.3.2 Unit Testing

Focused on validating business logic such as:

- ResumeMatchingService scoring logic
- Skill recalculation algorithms

7.3.3 Integration Testing

Validated interaction between:

- Authentication and role-based routing
- Job posting and application flow
- Certificate upload and verification workflow
- Mock interview scoring and skill update

7.3.4 Simulated Dataset Testing

Mock datasets were inserted into:

- profiles
- jobs
- student_certifications
- aptitude_attempts

This ensured realistic stress conditions and verified relational integrity.

7.3.5 User Acceptance Testing (UAT)

Simulated real-world usage by testing the platform from three perspectives:

- Student
- Recruiter
- College

7.4 Testing Steps

7.4.1 Unit Testing

Individual components were tested in isolation.

Example:

- ResumeMatchingService was tested with controlled keyword sets.
- Score accuracy validated up to two decimal precision.

7.4.2 Integration Testing

Validated multi-module workflows:

- Student applies for job → Match score generated → Recruiter sees ranked candidate.
- Certificate uploaded → Validation rule triggered → College approval updates trust score.
- Mock interview completed → Skill score recalculated → Dashboard reflects updated readiness.

7.4.3 Validation Testing

Ensured proper input handling:

- Email and password validation during signup.
- Mandatory job fields enforcement.
- Only supported resume formats accepted.
- Required course fields validation before publishing.

7.4.4 Output Testing

Verified correctness of generated outputs:

- Match Score percentage chip rendering.
- Placement analytics charts reflecting offers data.
- Resume PDF generation matching stored profile JSON.
- Mock interview feedback storage and display.

7.4.5 Security Testing

Special focus was given to:

- Student attempting to access College dashboard.
- Recruiter attempting to modify another organization's job.
- Direct database query simulation against RLS policies.

All unauthorized attempts were blocked.

7.5 Testing Cases

The following test cases were executed to validate the functional correctness, security enforcement, AI-driven analytics, real-time communication, and overall system stability of the ElevateHire platform. Each test case represents a practical scenario derived from real user workflows across Student, Recruiter, and College roles. The results confirm that the system behaves as expected under normal operating conditions.

Test Case ID	Feature	Description	Expected Result	Actual Result	Status
TC-01	Authentication	Login with valid Recruiter credentials	Redirect to Recruiter Dashboard	Redirected successfully	Pass
TC-02	AI Matcher	Upload resume with 5/10 matching keywords	System shows 50% match score	Shown 50.0%	Pass
TC-03	Security	Student attempts to access College Drive settings	Access denied via RLS and routing	Access Denied	Pass
TC-04	Job Post	Recruiter creates job without required salary field	Validation error displayed	Validation message shown	Pass
TC-05	Candidate Search	Search for "Flutter" skill	Relevant students listed	Listed 12 students	Pass
TC-06	Certificate Validation	Student uploads new certificate	Status set to Pending Verification	Status: Pending	Pass
TC-07	Analytics	Complete mock interview session	Skill proficiency updates immediately	Score updated	Pass
TC-08	Learning	Enroll in "Advanced Dart" course	Course added to My Courses list	Course enrolled	Pass
TC-09	Mock System	College creates Technical Round mock	Session visible in student Prep Arena	Session published	Pass
TC-10	Networking	Student follows Recruiter	Recruiter appears in Connections list	Followed successfully	Pass

TC-11	Feed	Post project update to campus feed	Visible to Connections	Post visible	Pass
TC-12	Resume Builder	Export resume using "Modern Clean" template	PDF matches structured profile data	PDF generated correctly	Pass
TC-13	Identity	Change profile picture	Avatar updates across all modules	Avatar updated	Pass
TC-14	Placement Drive	College closes active drive	Drive status updates to Completed	Status updated	Pass
TC-15	ElevateAI Navigation	Ask "How to post a job?" or "Practice arena"	AI provides guide and navigation option	Guide shown and navigated	Pass
TC-16	ElevateAI Search	Query specific student or course name	AI retrieves entity and provides link	Entity found and link offered	Pass
TC-17	Messaging	Send real-time message to connected user	Message appears instantly via Realtime	Message received successfully	Pass

Table 7.5: Test Cases**7.6 Testing Summary**

The testing phase confirmed that:

- RLS security prevents cross-role data exposure.
- Database integrity is preserved under concurrent operations.
- UI responsiveness is stable across major browsers.

No critical defects were found in the final testing cycle.

8. SCREENSHOTS

8.1 Introduction

This chapter presents some visual representations of the ElevateHire platform to demonstrate the implemented features and user interface across different modules. The screenshots provide evidence of functional implementation and system usability across Student, Recruiter, and College roles.

8.2 Unified Login Interface

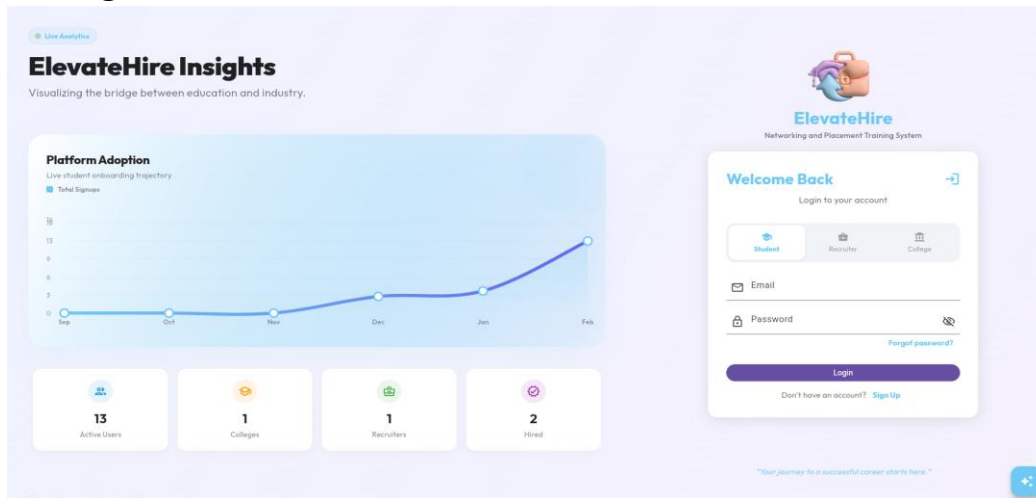


Figure 8.2: Unified Login Interface

8.3 Student Module

8.3.1 Student Dashboard

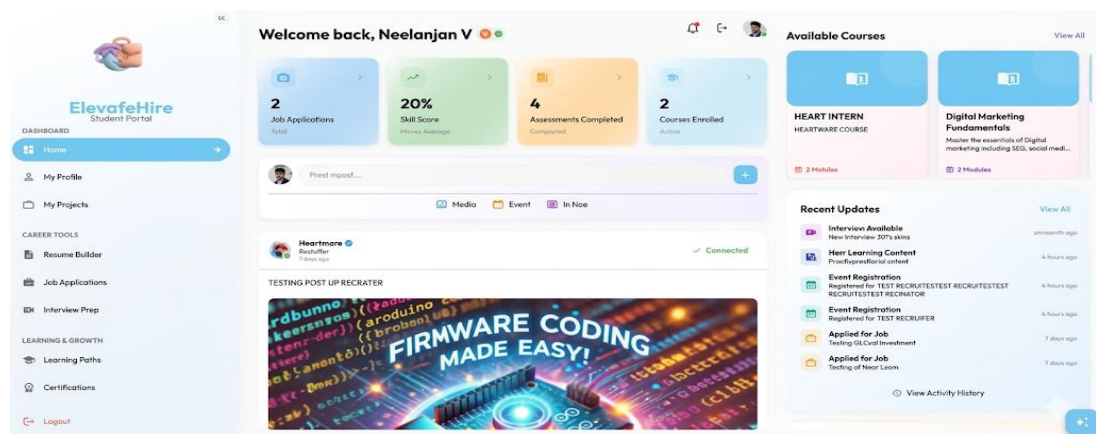


Figure 8.3: Student Dashboard

8.3.2 Project Portfolio

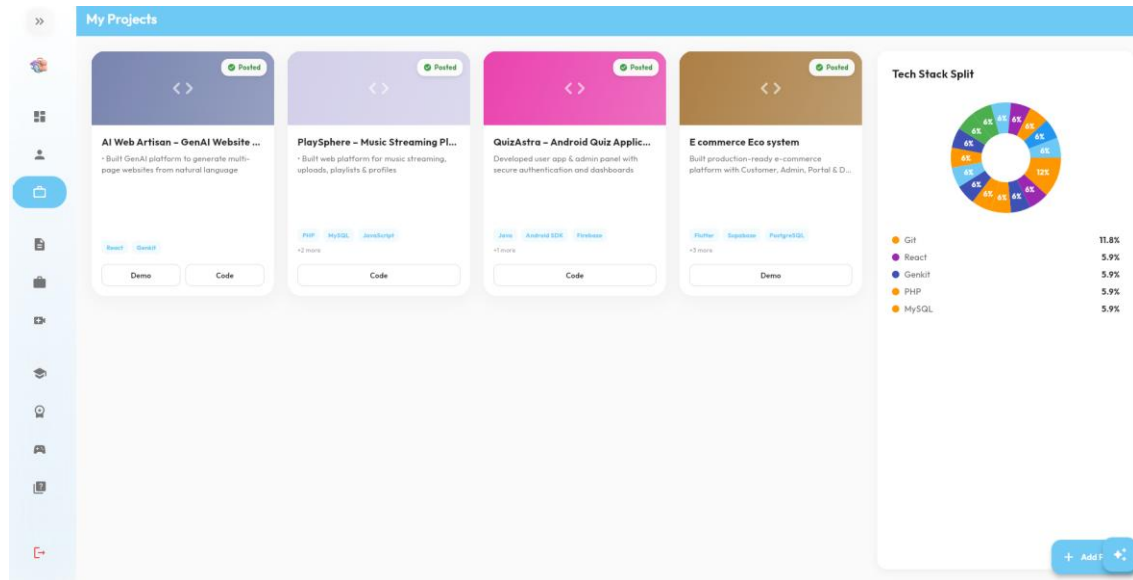


Figure 8.3.2: Project Portfolio

8.3.3 AI Resume Matching

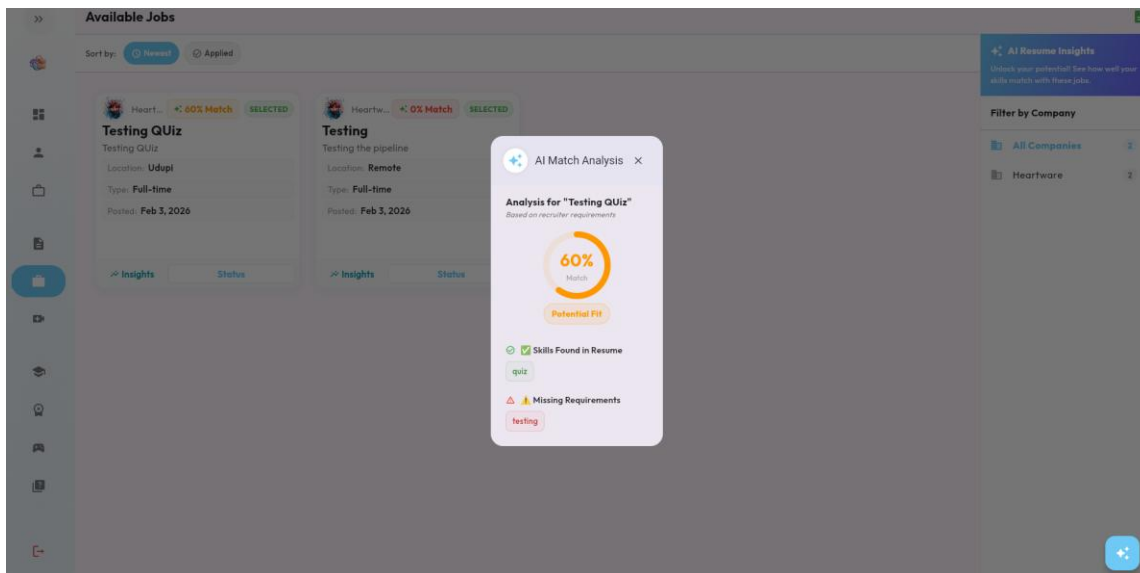


Figure8.3.3: AI Resume Matching Insights Dialog

8.3.4 Resume Builder

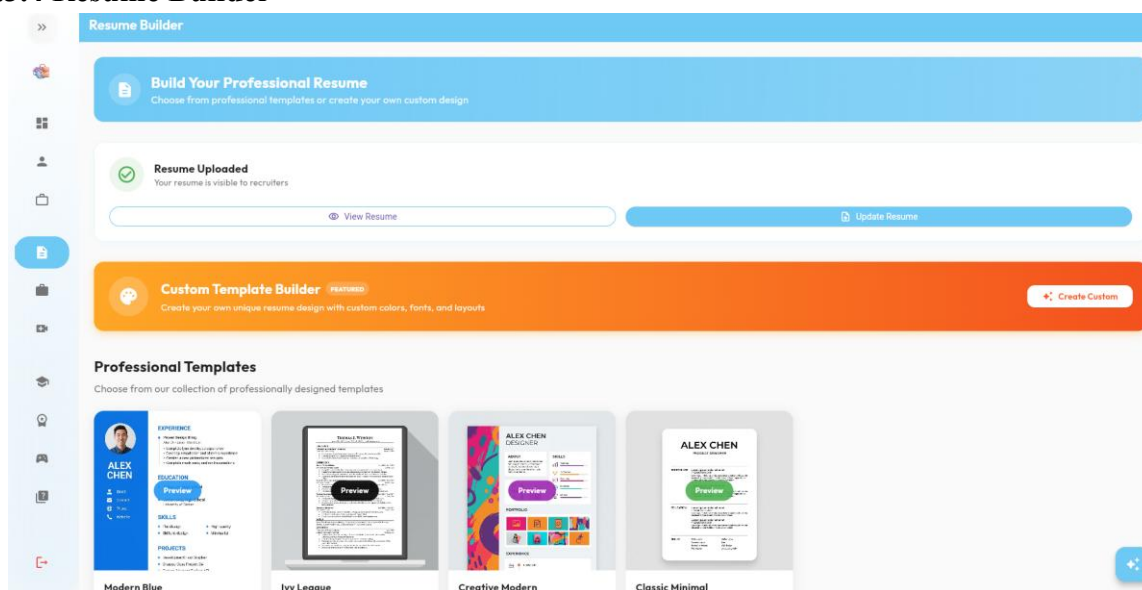


Figure 8.3.4: Resume Builder

8.3.5 Learning Hub

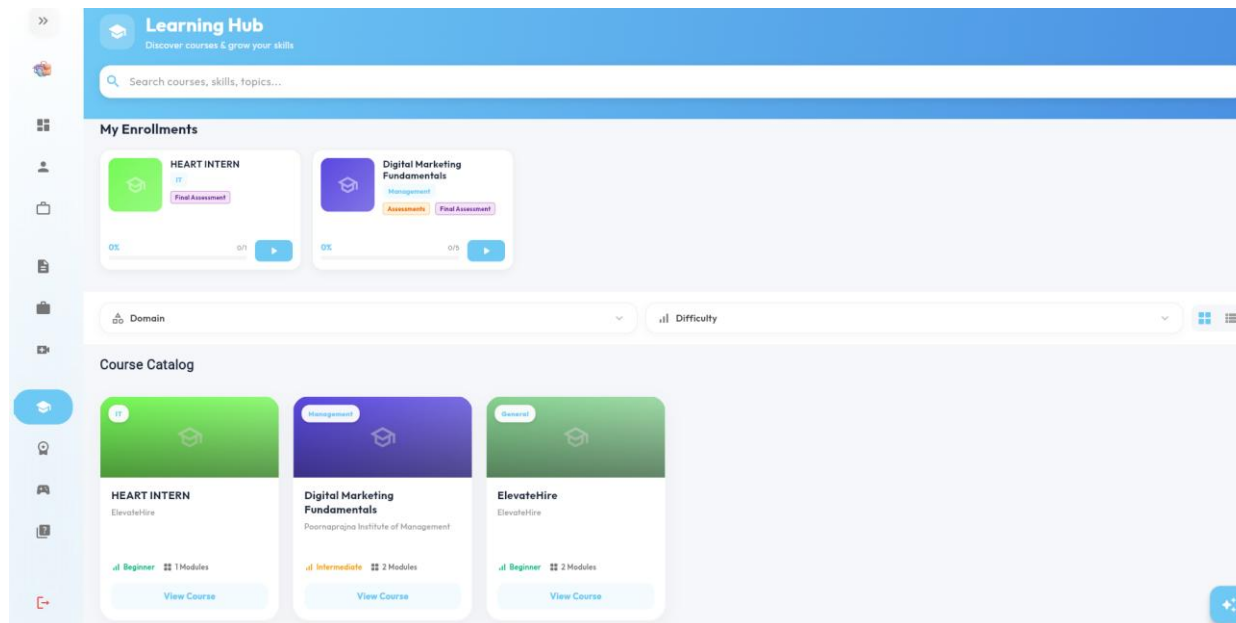


Figure 8.3.5 Learning Hub

8.4 College Module

8.4.1 College Dashboard

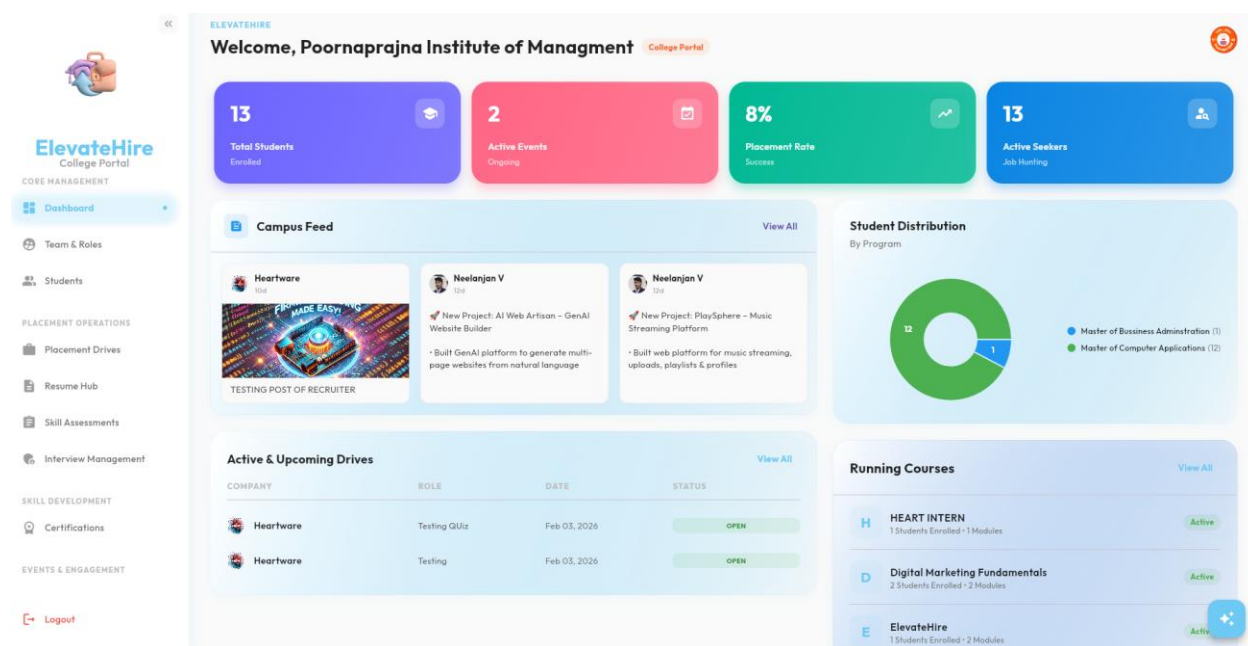


Figure 8.4.1 College Dashboard

8.4.2 Placement Tracking

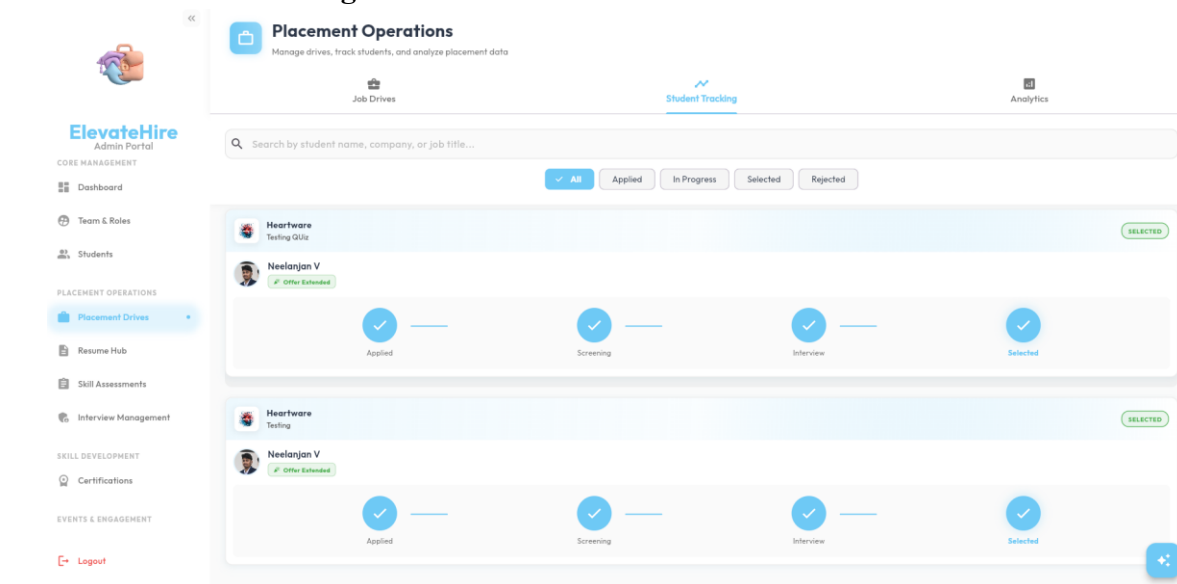


Figure 8.4.2:Placment Tracking

8.4.3 Analytics Insights

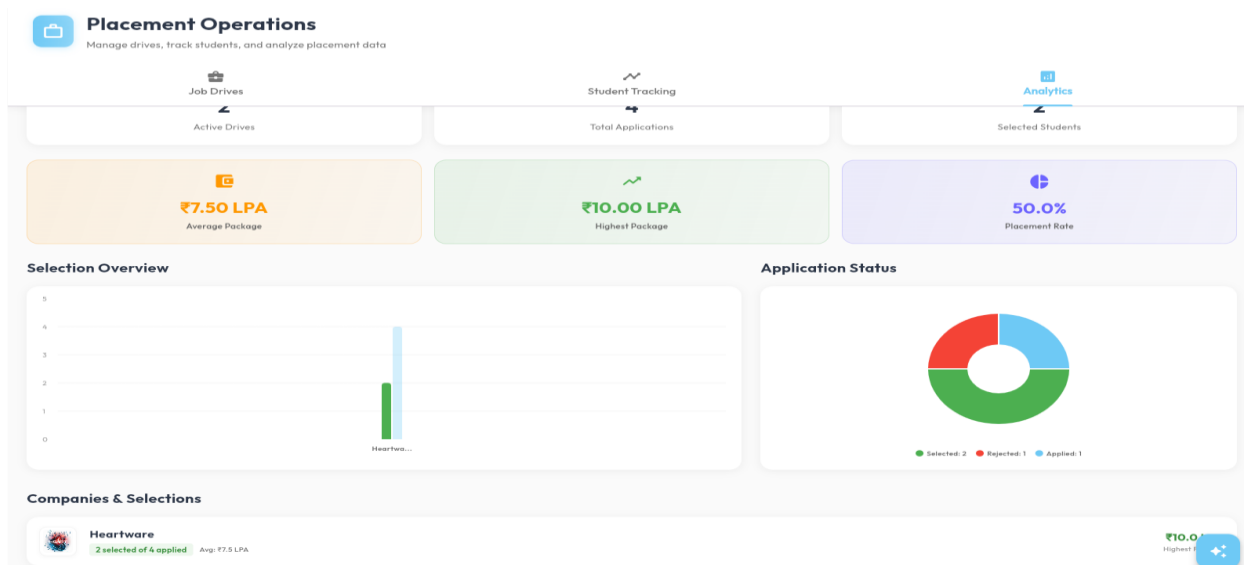


Figure 8.4.3: Analytics Insights

8.5 Recruiter Module

8.5.1 Recruitment Dashboard

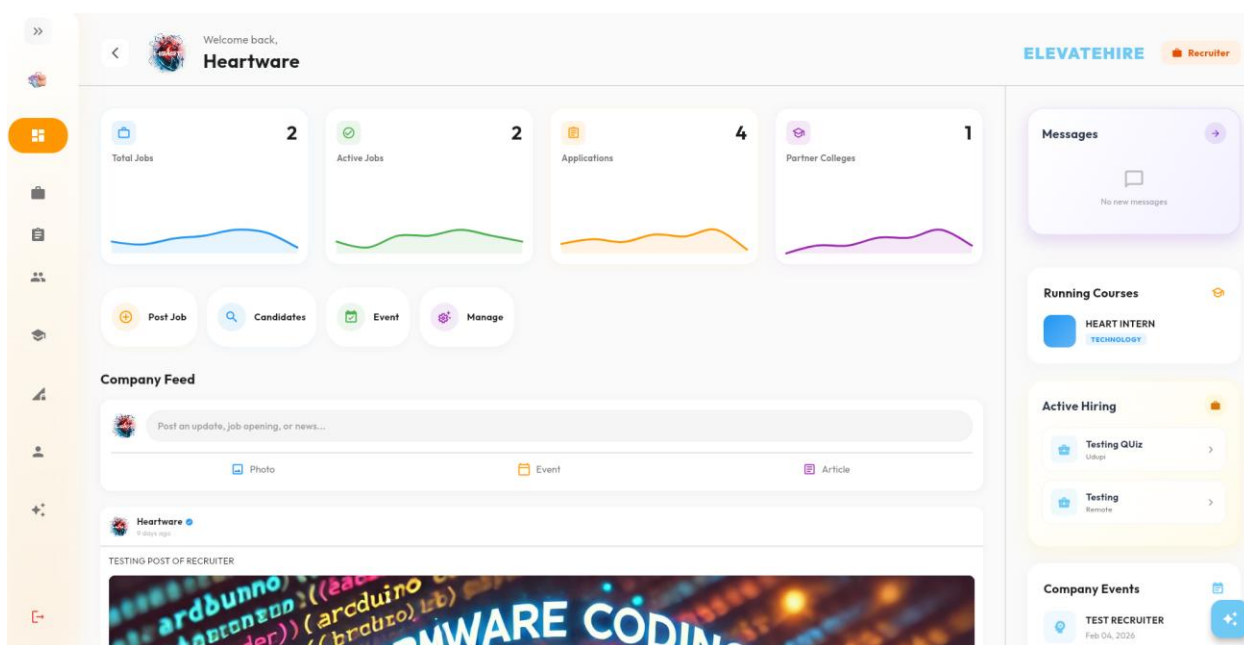


Figure 8.5.1: Recruitment Dashboard

8.5.2 Job Posting

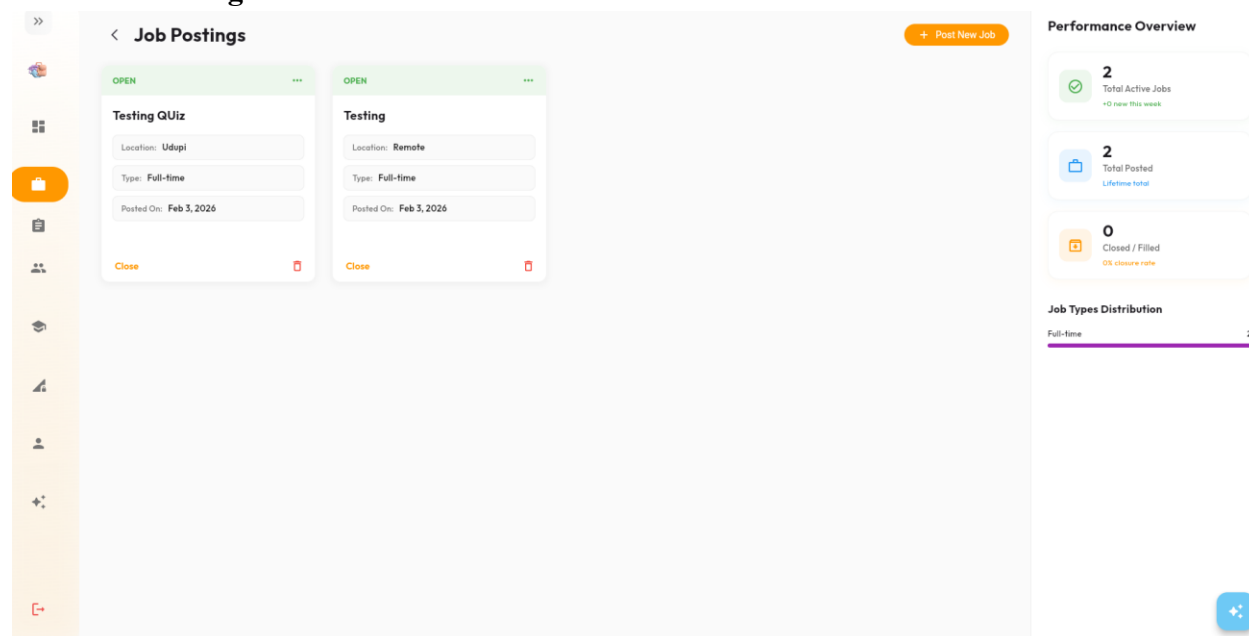


Figure 8.5.2 Job posting

8.5.3 Candidates

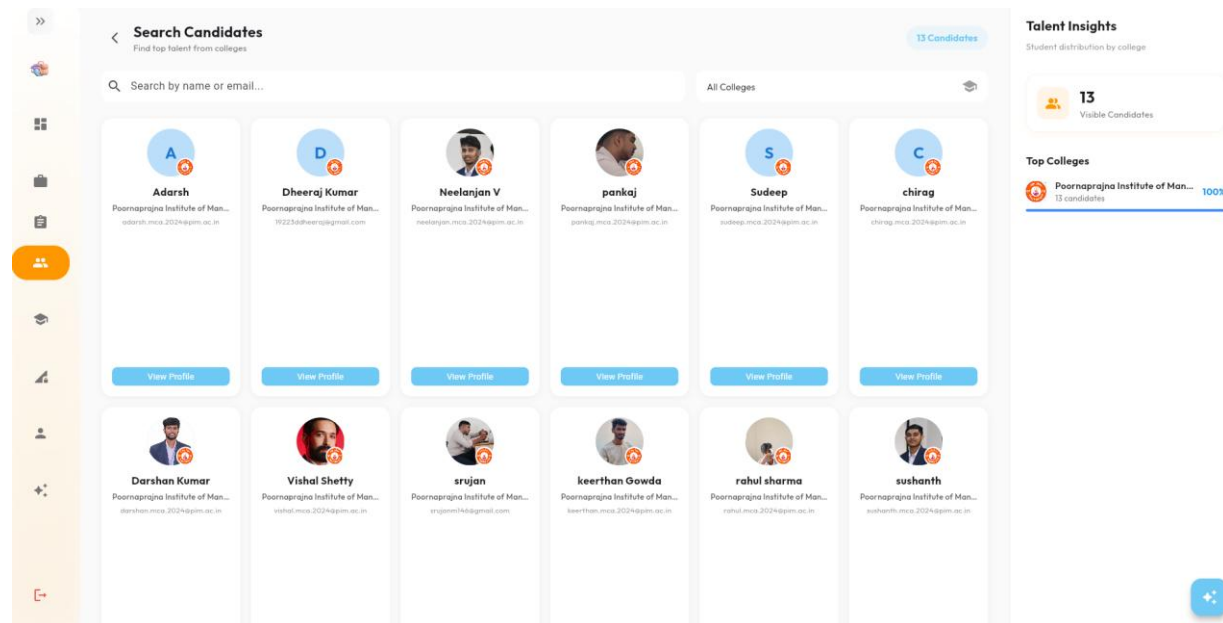


Figure 8.5.3 Candidates

8.6 ElevateAI-Beta

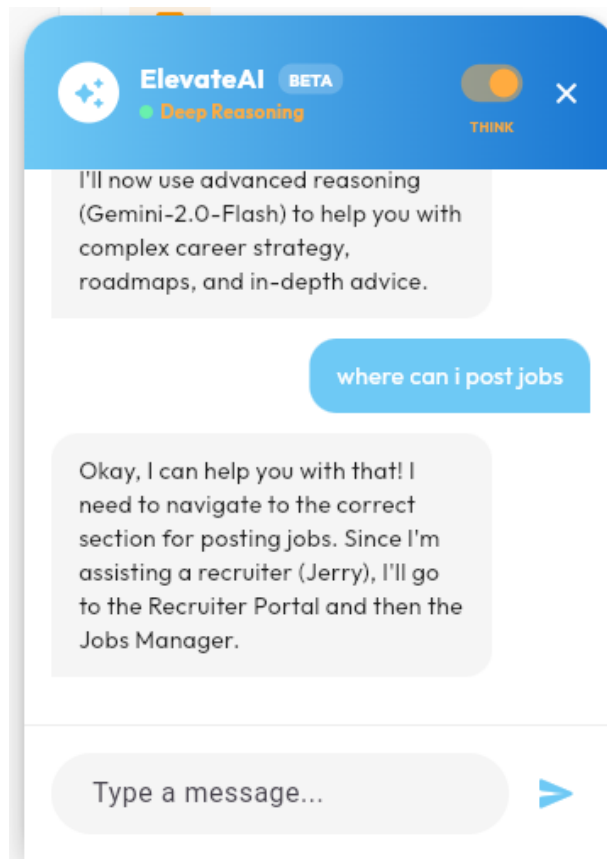


Figure 8.6 Elevate AI Beta Chatbot

8.7 Conclusion

The following section presents selected screenshots from the ElevateHire platform to visually demonstrate key features and user interfaces implemented for different user roles. These images highlight the login flow, student dashboard, recruiter analytics, college placement tools, networking feed, and AI Assistant interactions. They serve to validate the system's functionality and design consistency. For a complete experience of the working application, please visit the live deployment at: <https://elevate-hire-app.vercel.app/>.

9.CONCLUSION AND FUTURE ENHANCEMENT

9.1 Conclusion

The development of ElevateHire successfully addresses the need for a transparent and technology-driven placement ecosystem. By unifying students, recruiters, and colleges into a single digital platform, the system streamlines the transition from academics to professional careers.

The project highlights the impact of structured data management, modular architecture, and AI-assisted intelligence in improving recruitment workflows. Key achievements include a unified multi-role platform, an AI-powered resume matching engine, scalable real-time performance using Flutter and Supabase, institutional analytics and verification tools, and integrated professional networking features.

In summary, ElevateHire provides a strong technological foundation for modern recruitment, transforming traditional placement processes into a more efficient, data-driven, and merit-based system.

9.2 Future Enhancement

Although ElevateHire provides a comprehensive placement ecosystem, the platform is designed for continuous improvement. Future enhancements may include:

1. Advanced AI Integration – Incorporating Large Language Models for personalized resume suggestions, mock interview feedback, and career guidance.
2. Integrated Video Interview System – Enabling in-platform live interviews with AI-assisted monitoring and evaluation.
3. Blockchain Credential Verification – Implementing secure, tamper-proof academic and certification validation.
4. Corporate Training Expansion – Extending the platform to support onboarding and continuous employee upskilling.

These enhancements will strengthen ElevateHire’s scalability, intelligence, and global adoption potential.

10.BIBLIOGRAPHY

- [1] Flutter Developers. (n.d.). Flutter documentation. <https://docs.flutter.dev/>
- [2] Harrison, G. (2015). Next generation databases. Apress.
- [3] Martin, R. C. (2017). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.
- [4] Material Design. (n.d.). Material Design 3 guidelines. <https://m3.material.io/>
- [5] Microsoft. (n.d.). Visual Studio Code documentation. <https://code.visualstudio.com/docs>
- [6] PostgreSQL Global Development Group. (n.d.). PostgreSQL documentation. <https://www.postgresql.org/docs/>
- [7] Riverpod. (n.d.). Riverpod documentation. <https://riverpod.dev/>
- [8] Sommerville, I. (2012). Software engineering (9th ed.). Pearson Education.
- [9] Supabase. (n.d.). Supabase documentation. <https://supabase.com/docs>
- [10] The Dart Team. (n.d.). Dart language documentation. <https://dart.dev/guides>
- [11] Yourdon, E., & Constantine, L. L. (1979). Structured design: Fundamentals of a discipline of computer program and systems design. Prentice Hall.